

인공지능수학 기말프로젝트 — 코드

선형회귀 · 다중 클래스 로지스틱 회귀 · 딥러닝

학번: _____ 이름: _____

실행 환경: Python 3.10 · NumPy 2.2 · scikit-learn 1.7 · TensorFlow(CPU) 2.21
생성형 AI 활용 표기: 코드 작성과 디버깅에 Anthropic Claude를 활용하였음 (보고서 표기 참조)

프로젝트 1. 선형회귀 — Diamonds 가격 예측

파일: code/project1_linear_regression.py

```
1 | # -*- coding: utf-8 -*-
2 | """
3 | [프로젝트 1] 선형회귀 - Diamonds 데이터셋 가격 예측
4 | 데이터 출처: ggplot2 diamonds dataset (53,940개 다이아몬드의 가격과 특성)
5 |               pydataset 패키지를 통해 로드 (https://github.com/iamaziz/PyDataset)
6 |               원 출처: R ggplot2 패키지 (https://ggplot2.tidyverse.org/reference/diamonds.html)
7 |
8 | 목표: 다이아몬드의 물리적 특성(캐럿, 컷, 색상, 투명도, 치수)으로 가격(USD)을 예측
9 |
10 | 수업 이론 적용:
11 |   1) MSE(평균제곱오차) 손실함수
12 |   2) 정규방정식(Normal Equation)으로 해석적 최적해 계산:  $w = (X^T X)^{-1} X^T y$ 
13 |   3) 배치 경사하강법(Batch Gradient Descent)으로 수치적 최적해 계산
14 |   4) 두 방법과 scikit-learn LinearRegression 결과 비교 검증
15 |   5) 로그 변환을 통한 성능 개선 실험
16 | """
17 |
18 | import json
19 | import numpy as np
20 | import pandas as pd
21 | import matplotlib
22 | matplotlib.use("Agg") # 화면 없이 그림 파일만 저장
23 | import matplotlib.pyplot as plt
24 | from pydataset import data
25 | from sklearn.model_selection import train_test_split
26 | from sklearn.linear_model import LinearRegression
27 | from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
28 |
29 | RNG_SEED = 42 # 재현성을 위해 난수 시드 고정
30 | np.random.seed(RNG_SEED)
31 |
32 | FIG_DIR = "figures"
33 | RES_DIR = "results"
34 |
35 | # -----
36 | # 1. 데이터 로드 및 탐색 (EDA)
37 | # -----
38 | df = data("diamonds")
39 | print("데이터 크기:", df.shape) # (53940, 10)
40 | print(df.head())
41 | print(df.describe())
42 | print("결측치 개수:\n", df.isna().sum())
43 |
44 | # x, y, z(길이/너비/깊이 mm)가 0인 행은 물리적으로 불가능한 오류 데이터이므로 제거
45 | # 또한 y=58.9mm, z=31.8mm 같은 기록 오류 수준의 극단값도 제거
46 | # (처음에는 0만 제거했는데, 극단값 때문에 로그 모델의 exp 복원이 폭주하는 문제를 겪고 기준을 추가함)
47 | n_before = len(df)
48 | df = df[(df["x"] > 0) & (df["y"] > 0) & (df["z"] > 0)]
49 | df = df[(df["y"] < 20) & (df["z"] < 20)]
50 | print(f"이상치 제거: {n_before} -> {len(df)} 행")
51 |
52 | # 가격 분포 시각화: 오른쪽으로 길게 치우친(skewed) 분포 확인
53 | fig, axes = plt.subplots(1, 2, figsize=(11, 4))
54 | axes[0].hist(df["price"], bins=80, color="steelblue", edgecolor="none")
55 | axes[0].set_title("Price distribution (skewed)")
56 | axes[0].set_xlabel("price (USD)"); axes[0].set_ylabel("count")
57 | axes[1].scatter(df["carat"], df["price"], s=2, alpha=0.15, color="darkorange")
58 | axes[1].set_title("Carat vs Price (nonlinear)")
59 | axes[1].set_xlabel("carat"); axes[1].set_ylabel("price (USD)")
60 | plt.tight_layout()
61 | plt.savefig(f"{FIG_DIR}/p1_eda.png", dpi=140)
62 | plt.close()
63 |
64 | # -----
65 | # 2. 전처리
66 | # -----
67 | # cut, color, clarity는 등급에 순서가 있는 범주형 변수(ordinal)
68 | # -> 품질이 좋을수록 큰 경수가 되도록 직접 매핑 (도메인 지식 반영)
```

```

69 | cut_order    = {"Fair": 0, "Good": 1, "Very Good": 2, "Premium": 3, "Ideal": 4}
70 | color_order  = {c: i for i, c in enumerate(["J", "I", "H", "G", "F", "E", "D"])} # J(나쁨)->D(좋음)
71 | clarity_order = {c: i for i, c in enumerate(["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"])}
72 |
73 | df["cut_o"]   = df["cut"].map(cut_order)
74 | df["color_o"] = df["color"].map(color_order)
75 | df["clarity_o"] = df["clarity"].map(clarity_order)
76 |
77 | feature_cols = ["carat", "cut_o", "color_o", "clarity_o", "depth", "table", "x", "y", "z"]
78 | X = df[feature_cols].to_numpy(dtype=float)
79 | y = df["price"].to_numpy(dtype=float)
80 |
81 | # 학습/테스트 분리 (8:2). 테스트셋은 학습에 일절 사용하지 않음
82 | X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=RNG_SEED)
83 |
84 | # 표준화(standardization): 각 특성을 평균 0, 표준편차 1로 변환
85 | # 경사하강법에서 특성 간 스케일 차이가 크면 손실함수 등고선이 길쭉해져 수렴이 느려지므로 필수
86 | mu, sigma = X_tr.mean(axis=0), X_tr.std(axis=0)
87 | X_tr_s = (X_tr - mu) / sigma
88 | X_te_s = (X_te - mu) / sigma # 테스트셋도 '학습셋의' 통계량으로 변환 (데이터 누수 방지)
89 |
90 | # 편향(bias) 항을 위한 1 열 추가 -> 행렬 한 번의 곱으로  $w_0 + w \cdot x$  계산 가능
91 | ones_tr = np.ones((X_tr_s.shape[0], 1))
92 | ones_te = np.ones((X_te_s.shape[0], 1))
93 | Xb_tr = np.hstack([ones_tr, X_tr_s])
94 | Xb_te = np.hstack([ones_te, X_te_s])
95 |
96 | # -----
97 | # 3. 방법 1: 정규방정식 (해석적 해)
98 | # MSE를  $w$ 로 미분해 0으로 두면  $w^* = (X^T X)^{-1} X^T y$ 
99 | # -----
100 | w_ne = np.linalg.inv(Xb_tr.T @ Xb_tr) @ Xb_tr.T @ y_tr
101 | pred_ne_te = Xb_te @ w_ne
102 |
103 | # -----
104 | # 4. 방법 2: 배치 경사하강법 (수치적 해)
105 | #  $L(w) = (1/N) \sum (y_i - w \cdot x_i)^2$ 
106 | #  $\nabla L = (2/N) X^T (Xw - y)$ ,  $w \leftarrow w - lr * \nabla L$ 
107 | # -----
108 | def gradient_descent(Xb, y, lr=0.1, epochs=300):
109 |     N, d = Xb.shape
110 |     w = np.zeros(d) # 가중치 0으로 초기화
111 |     history = [] # epoch별 MSE 기록 (수렴 확인용)
112 |     for ep in range(epochs):
113 |         err = Xb @ w - y # 예측 - 정답
114 |         grad = (2.0 / N) * (Xb.T @ err) # MSE의 기울기 벡터
115 |         w -= lr * grad # 기울기 반대 방향으로 한 걸음
116 |         history.append(float(np.mean(err ** 2)))
117 |     return w, history
118 |
119 | w_gd, hist = gradient_descent(Xb_tr, y_tr, lr=0.1, epochs=300)
120 | pred_gd_te = Xb_te @ w_gd
121 |
122 | # 학습률 비교 실험: 너무 작으면 느리고, 너무 크면 발산함을 확인
123 | lr_exp = {}
124 | for lr in [0.001, 0.01, 0.1, 1.0]:
125 |     _, h = gradient_descent(Xb_tr, y_tr, lr=lr, epochs=300)
126 |     lr_exp[lr] = h
127 |
128 | plt.figure(figsize=(7, 4.5))
129 | CAP = 1e9 # 발산한 손실은 그림 범위를 위해 1e9까지만 표시
130 | for lr, h in lr_exp.items():
131 |     h = np.asarray(h)
132 |     ok = np.isfinite(h) & (h < CAP) # lr=1.0은 손실이 inf로 폭주하므로 범위 내 값만 그림
133 |     n_ok = int(ok.sum())
134 |     label = f"lr={lr}" + (" if n_ok == len(h) else " (diverges)")
135 |     plt.plot(np.arange(len(h))[ok], h[ok], label=label)
136 | plt.yscale("log")
137 | plt.xlabel("epoch"); plt.ylabel("train MSE (log scale)")
138 | plt.title("Gradient descent convergence by learning rate")
139 | plt.legend(); plt.tight_layout()
140 | plt.savefig(f"{FIG_DIR}/p1_lr_compare.png", dpi=140)
141 | plt.close()

```

```

142 |
143 | # -----
144 | # 5. 방법 3: scikit-learn LinearRegression (검증용 기준)
145 | # -----
146 | sk = LinearRegression().fit(X_tr_s, y_tr)
147 | pred_sk_te = sk.predict(X_te_s)
148 |
149 | # 세 방법의 가중치가 (거의) 일치하는지 확인 -> 구현이 올바른다는 근거
150 | print("정규방정식 w[:4]:", np.round(w_ne[:4], 3))
151 | print("경사하강법 w[:4]:", np.round(w_gd[:4], 3))
152 | print("sklearn w[:4]:", np.round(np.r_[sk.intercept_, sk.coef_[:3]], 3))
153 |
154 | # -----
155 | # 6. 개선 실험: 목표값 로그 변환
156 | #   가격 분포가 한쪽으로 치우쳐 있고 carat-price 관계가 비선형이므로
157 | #   log(price)를 예측하면 선형 모형의 가정에 더 가까워진다
158 | # -----
159 | ylog_tr, ylog_te = np.log(y_tr), np.log(y_te)
160 | w_log = np.linalg.inv(Xb_tr.T @ Xb_tr) @ Xb_tr.T @ ylog_tr
161 | pred_log_space = Xb_te @ w_log          # 로그 공간에서의 예측값
162 | pred_log_te = np.exp(pred_log_space)    # 로그 공간 예측을 다시 달러로 환원
163 | r2_log_space = r2_score(ylog_te, pred_log_space) # 로그 공간 자체의 설명력도 따로 기록
164 |
165 | # -----
166 | # 7. 평가 및 결과 저장
167 | # -----
168 | def metrics(y_true, y_pred):
169 |     return {
170 |         "MSE": float(mean_squared_error(y_true, y_pred)),
171 |         "RMSE": float(np.sqrt(mean_squared_error(y_true, y_pred))),
172 |         "MAE": float(mean_absolute_error(y_true, y_pred)),
173 |         "R2": float(r2_score(y_true, y_pred)),
174 |     }
175 |
176 | results = {
177 |     "n_samples": int(len(df)),
178 |     "n_features": len(feature_cols),
179 |     "normal_equation": metrics(y_te, pred_ne_te),
180 |     "gradient_descent": metrics(y_te, pred_gd_te),
181 |     "sklearn": metrics(y_te, pred_sk_te),
182 |     "log_target": metrics(y_te, pred_log_te),
183 |     "log_target_logspace_R2": float(r2_log_space),
184 |     "weights_ne": {c: float(w) for c, w in zip(["bias"] + feature_cols, w_ne)},
185 |     "gd_final_train_mse": hist[-1],
186 | }
187 | print(json.dumps(results, indent=2))
188 | with open(f"{RES_DIR}/p1_results.json", "w") as f:
189 |     json.dump(results, f, indent=2)
190 |
191 | # 예측 vs 실제, 잔차 그림
192 | fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
193 | axes[0].scatter(y_te, pred_ne_te, s=2, alpha=0.15)
194 | lim = [0, y_te.max()]
195 | axes[0].plot(lim, lim, "r--", lw=1)
196 | axes[0].set_xlabel("actual price"); axes[0].set_ylabel("predicted price")
197 | axes[0].set_title(f"Linear model (R2={results['normal_equation']['R2']:.3f})")
198 | axes[1].scatter(y_te, pred_log_te, s=2, alpha=0.15, color="green")
199 | axes[1].plot(lim, lim, "r--", lw=1)
200 | axes[1].set_xlabel("actual price"); axes[1].set_ylabel("predicted price")
201 | axes[1].set_title(f"Log-target model (R2={results['log_target']['R2']:.3f})")
202 | plt.tight_layout()
203 | plt.savefig(f"{FIG_DIR}/p1_pred_vs_actual.png", dpi=140)
204 | plt.close()
205 |
206 | # 잔차(residual) 분석: 잔차가 0 주위에 고르게 퍼져 있어야 좋은 모형
207 | plt.figure(figsize=(7, 4.5))
208 | plt.scatter(pred_ne_te, y_te - pred_ne_te, s=2, alpha=0.15)
209 | plt.axhline(0, color="r", ls="--", lw=1)
210 | plt.xlabel("predicted price"); plt.ylabel("residual (actual - predicted)")
211 | plt.title("Residual plot (heteroscedasticity visible)")
212 | plt.tight_layout()
213 | plt.savefig(f"{FIG_DIR}/p1_residual.png", dpi=140)
214 | plt.close()

```

```
215 |  
216 | print("프로젝트 1 완료")  
217 |
```

프로젝트 2. 다중 클래스 로지스틱 회귀 — Wine 품종 분류

파일: code/project2_softmax_regression.py

```
1 | # -*- coding: utf-8 -*-
2 | """
3 | [프로젝트 2] 다중 클래스 로지스틱 회귀 (소프트맥스 회귀) - Wine 데이터셋
4 | 데이터 출처: UCI Wine dataset (scikit-learn 내장 load_wine 으로 로드)
5 |             https://archive.ics.uci.edu/ml/datasets/wine
6 |             이탈리아 같은 지역에서 재배된 3개 품종(cultivar)의 와인 178개,
7 |             화학 성분 분석 13개 특성 (알코올, 말산, 플라보노이드, 색 강도 등)
8 |
9 | 목표: 13개 화학 성분 측정값으로 와인 품종(3 클래스)을 분류
10 |
11 | 수업 이론 적용:
12 | 1) 소프트맥스 함수로 선형 점수 -> 클래스 확률 변환
13 | 2) 원-핫 인코딩과 크로스 엔트로피 손실
14 | 3) 손실의 기울기  $\partial L / \partial W = X^T (P - Y) / N$  를 직접 유도한 식으로 구현
15 | 4) 배치 경사하강법 학습, 학습률/표준화 영향 실험
16 | 5) scikit-learn LogisticRegression(다중 클래스)과 비교 검증
17 | """
18 |
19 | import json
20 | import numpy as np
21 | import matplotlib
22 | matplotlib.use("Agg")
23 | import matplotlib.pyplot as plt
24 | from sklearn.datasets import load_wine
25 | from sklearn.model_selection import train_test_split
26 | from sklearn.linear_model import LogisticRegression
27 | from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
28 |
29 | RNG_SEED = 42
30 | np.random.seed(RNG_SEED)
31 | FIG_DIR, RES_DIR = "figures", "results"
32 |
33 | # -----
34 | # 1. 데이터 로드 및 탐색
35 | # -----
36 | wine = load_wine()
37 | X, y = wine.data, wine.target          # X: (178, 13), y: 0/1/2
38 | print("특성 이름:", wine.feature_names)
39 | print("클래스별 개수:", np.bincount(y))    # [59, 71, 48] -> 약간의 불균형
40 | print("특성별 평균:", np.round(X.mean(axis=0), 2))
41 | print("특성별 표준편차:", np.round(X.std(axis=0), 2))
42 | # proline의 표준편차(약 314)와 nonflavanoid_phenols(약 0.12)가 2600배 이상 차이
43 | # -> 표준화 없이는 경사하강법이 제대로 동작하기 어려움을 예상할 수 있다
44 |
45 | # 클래스가 고루 섞이도록 stratify 적용하여 8:2 분리
46 | X_tr, X_te, y_tr, y_te = train_test_split(
47 |     X, y, test_size=0.2, random_state=RNG_SEED, stratify=y)
48 |
49 | # 표준화 (학습셋 통계량만 사용)
50 | mu, sigma = X_tr.mean(axis=0), X_tr.std(axis=0)
51 | X_tr_s = (X_tr - mu) / sigma
52 | X_te_s = (X_te - mu) / sigma
53 |
54 | K = 3                                # 클래스 수
55 | N, D = X_tr_s.shape
56 |
57 | # -----
58 | # 2. 소프트맥스 회귀 직접 구현
59 | # -----
60 | def one_hot(y, K):
61 |     """경답 레이블을 원-핫 벡터로 변환. 예: 1 -> [0,1,0]"""
62 |     Y = np.zeros((len(y), K))
63 |     Y[np.arange(len(y)), y] = 1.0
64 |     return Y
65 |
66 | def softmax(U):
67 |     """행 단위 소프트맥스. 행별 최대값을 빼서 exp 오버플로 방지(수치 안정화)"""
68 |     U = U - U.max(axis=1, keepdims=True)
```

```

69 |     E = np.exp(U)
70 |     return E / E.sum(axis=1, keepdims=True)
71 |
72 | def cross_entropy(P, Y):
73 |     """평균 크로스 엔트로피. log(0) 방지를 위해 아주 작은 값을 더함"""
74 |     return float(-np.mean(np.sum(Y * np.log(P + 1e-12), axis=1)))
75 |
76 | def train_softmax(X, y, K, lr=0.1, epochs=500):
77 |     """배치 경사하강법으로 소프트맥스 회귀 학습.
78 |     U = XW + b, P = softmax(U), L = CE(P, Y)
79 |     ∂L/∂W = X^T (P - Y) / N <- 수업에서 chain rule로 유도한 식
80 |     ∂L/∂b = 평균(P - Y)
81 |     """
82 |     N, D = X.shape
83 |     Y = one_hot(y, K)
84 |     W = np.zeros((D, K))          # 가중치 행렬 (13 x 3)
85 |     b = np.zeros(K)              # 편향 벡터
86 |     loss_hist, acc_hist = [], []
87 |     for ep in range(epochs):
88 |         P = softmax(X @ W + b)    # (N, 3) 각 행이 클래스 확률
89 |         loss_hist.append(cross_entropy(P, Y))
90 |         acc_hist.append(float(np.mean(P.argmax(axis=1) == y)))
91 |         G = (P - Y) / N           # 공통 항
92 |         W -= lr * (X.T @ G)        # 가중치 갱신
93 |         b -= lr * G.sum(axis=0)    # 편향 갱신
94 |     return W, b, loss_hist, acc_hist
95 |
96 | W, b, loss_hist, acc_hist = train_softmax(X_tr_s, y_tr, K, lr=0.5, epochs=500)
97 |
98 | # 테스트 정확도
99 | P_te = softmax(X_te_s @ W + b)
100 | pred_te = P_te.argmax(axis=1)
101 | acc_scratch = accuracy_score(y_te, pred_te)
102 | print(f"직접 구현 테스트 정확도: {acc_scratch:.4f}")
103 | print("혼동행렬:\n", confusion_matrix(y_te, pred_te))
104 | print(classification_report(y_te, pred_te, target_names=wine.target_names))
105 |
106 | # -----
107 | # 3. 실험 1: 표준화를 하지 않으면?
108 | # -----
109 | W_r, b_r, loss_raw, acc_raw = train_softmax(X_tr, y_tr, K, lr=0.5, epochs=500)
110 | pred_raw = softmax(X_te @ W_r + b_r).argmax(axis=1)
111 | acc_no_std = accuracy_score(y_te, pred_raw)
112 | print(f"표준화 없이 학습한 테스트 정확도: {acc_no_std:.4f}")
113 |
114 | # 작은 학습률로 다시 시도 (발산은 피하지만 수렴이 매우 느림을 확인)
115 | W_r2, b_r2, loss_raw2, _ = train_softmax(X_tr, y_tr, K, lr=1e-6, epochs=500)
116 | pred_raw2 = softmax(X_te @ W_r2 + b_r2).argmax(axis=1)
117 | acc_no_std_small_lr = accuracy_score(y_te, pred_raw2)
118 | print(f"표준화 없음 + 작은 학습률 정확도: {acc_no_std_small_lr:.4f}")
119 |
120 | # -----
121 | # 4. 실험 2: 학습률에 따른 수렴 속도
122 | # -----
123 | plt.figure(figsize=(7, 4.5))
124 | for lr in [0.01, 0.1, 0.5, 2.0]:
125 |     _, _, h, _ = train_softmax(X_tr_s, y_tr, K, lr=lr, epochs=500)
126 |     plt.plot(h, label=f"lr={lr}")
127 | plt.xlabel("epoch"); plt.ylabel("train cross-entropy")
128 | plt.title("Convergence by learning rate (standardized)")
129 |
130 | plt.legend(); plt.tight_layout()
131 | plt.savefig(f"{FIG_DIR}/p2_lr_compare.png", dpi=140)
132 | plt.close()
133 |
134 | # 학습 곡선 (손실 + 정확도)
135 | fig, ax1 = plt.subplots(figsize=(7, 4.5))
136 | ax1.plot(loss_hist, color="tab:red", label="train CE loss")
137 | ax1.set_xlabel("epoch"); ax1.set_ylabel("cross-entropy", color="tab:red")
138 | ax2 = ax1.twinx()
139 | ax2.plot(acc_hist, color="tab:blue", label="train accuracy")
140 | ax2.set_ylabel("accuracy", color="tab:blue")
141 | plt.title("Softmax regression training curve")
142 | plt.tight_layout()

```

```

142 | plt.savefig(f"{FIG_DIR}/p2_training_curve.png", dpi=140)
143 | plt.close()
144 |
145 | # -----
146 | # 5. scikit-learn 다중 클래스 로지스틱 회귀와 비교
147 | # -----
148 | sk = LogisticRegression(max_iter=5000).fit(X_tr_s, y_tr)
149 | acc_sk = accuracy_score(y_te, sk.predict(X_te_s))
150 | print(f"sklearn 테스트 정확도: {acc_sk:.4f}")
151 |
152 | # -----
153 | # 6. 결정경계 시각화 (특성 2개만 사용한 보조 모델)
154 | #     13차원 전체는 그릴 수 없으므로, 판별력이 좋은 2개 특성만으로
155 | #     수업 자료(2차원 평면의 결정경계)와 같은 방식의 그림을 그린다
156 | # -----
157 | f1, f2 = 0, 6          # alcohol, flavanoids
158 | X2_tr = X_tr_s[:, [f1, f2]]
159 | X2_te = X_te_s[:, [f1, f2]]
160 | W2, b2, _, _ = train_softmax(X2_tr, y_tr, K, lr=0.5, epochs=1000)
161 | acc_2feat = accuracy_score(y_te, softmax(X2_te @ W2 + b2).argmax(axis=1))
162 |
163 | xx, yy = np.meshgrid(np.linspace(-3, 3, 300), np.linspace(-3, 3, 300))
164 | grid = np.c_[xx.ravel(), yy.ravel()]
165 | Z = softmax(grid @ W2 + b2).argmax(axis=1).reshape(xx.shape)
166 |
167 | plt.figure(figsize=(7, 5.5))
168 | plt.contourf(xx, yy, Z, alpha=0.25, levels=2, cmap="viridis")
169 | for k, name in enumerate(wine.target_names):
170 |     m = y_tr == k
171 |     plt.scatter(X2_tr[m, 0], X2_tr[m, 1], s=18, label=name)
172 | plt.xlabel("alcohol (standardized)"); plt.ylabel("flavanoids (standardized)")
173 | plt.title(f"Decision boundary with 2 features (test acc={acc_2feat:.3f})")
174 | plt.legend(); plt.tight_layout()
175 | plt.savefig(f"{FIG_DIR}/p2_decision_boundary.png", dpi=140)
176 | plt.close()
177 |
178 | # 혼동행렬 그림
179 | cm = confusion_matrix(y_te, pred_te)
180 | plt.figure(figsize=(5, 4.5))
181 | plt.imshow(cm, cmap="Blues")
182 | plt.colorbar()
183 | for i in range(K):
184 |     for j in range(K):
185 |         plt.text(j, i, cm[i, j], ha="center", va="center",
186 |                  color="white" if cm[i, j] > cm.max()/2 else "black")
187 | plt.xticks(range(K), wine.target_names); plt.yticks(range(K), wine.target_names)
188 | plt.xlabel("predicted"); plt.ylabel("actual")
189 | plt.title("Confusion matrix (scratch model)")
190 | plt.tight_layout()
191 | plt.savefig(f"{FIG_DIR}/p2_confusion.png", dpi=140)
192 | plt.close()
193 |
194 | # -----
195 | # 7. 결과 저장
196 | # -----
197 | results = {
198 |     "n_samples": int(X.shape[0]),
199 |     "n_features": int(X.shape[1]),
200 |     "class_counts": np.bincount(y).tolist(),
201 |     "acc_scratch": float(acc_scratch),
202 |     "acc_sklearn": float(acc_sk),
203 |     "acc_no_standardization": float(acc_no_std),
204 |     "acc_no_std_small_lr": float(acc_no_std_small_lr),
205 |     "acc_2features": float(acc_2feat),
206 |     "final_train_loss": loss_hist[-1],
207 |     "final_train_acc": acc_hist[-1],
208 |     "confusion_matrix": cm.tolist(),
209 | }
210 | print(json.dumps(results, indent=2))
211 | with open(f"{RES_DIR}/p2_results.json", "w") as f:
212 |     json.dump(results, f, indent=2)
213 |
214 | print("프로젝트 2 완료")

```


프로젝트 3. 딥러닝 — InstEval 강의평가 점수 예측

파일: code/project3_deep_learning.py

```
1 | # -*- coding: utf-8 -*-
2 | """
3 | [프로젝트 3] 딥러닝 - 강의평가 점수 예측 (InstEval, 5-클래스 분류)
4 | 데이터 출처: InstEval - ETH Zurich 강의평가 데이터 (R lme4 패키지 공개 데이터)
5 |               pydataset 패키지로 로드 (https://github.com/iamaziz/PyDataset)
6 |               원 출처: Bates et al., lme4 (https://cran.r-project.org/package=lme4)
7 |               73,421건의 강의평가. 학생 2,972명이 교수 1,128명의 강의에 1~5점을 매긴 기록
8 |
9 | 목표: (학생 ID, 교수 ID, 학생 연차, 수강 시점, 필수과목 여부, 학과)로부터
10 |       그 학생이 그 강의에 줄 평점(1~5)을 예측한다.
11 |
12 | 이 문제가 어려운 이유:
13 | - 입력의 핵심이 '누가, 누구를'이라는 초고차원 희소 범주형 변수(합계 4,100개 ID)
14 | - 같은 교수의 같은 강의도 학생마다 점수가 갈리는, 사람 평가 특유의 노이즈
15 | - 단순 다수결(최빈 클래스)은 약 24%: 이를 얼마나 넘어서는지가 관건
16 |
17 | 수업 이론 적용:
18 | 1) 원-핫 인코딩의 한계와 임베딩(embedding): 원-핫 벡터 x 가중치 행렬 = 행 선택임을 이용,
19 |     4,100차원 희소 표현을 16차원 밀집 벡터로 학습 (딥러닝의 표현 학습)
20 | 2) 다층 퍼셉트론 + ReLU + 드롭아웃 + Adam (week 13 / Keras 보조자료)
21 | 3) 소프트맥스 + 크로스 엔트로피 (프로젝트 2와 동일한 출력층 이론)
22 | 4) 깊은 망에서 ReLU vs 시그모이드 비교 (기울기 소실 확인 실험)
23 | 구현: Keras (TensorFlow 백엔드)
24 | """
25 |
26 | import json
27 | import pathlib
28 | import numpy as np
29 | import matplotlib
30 | matplotlib.use("Agg")
31 | import matplotlib.pyplot as plt
32 | from pydataset import data
33 | from sklearn.model_selection import train_test_split
34 | from sklearn.metrics import confusion_matrix, mean_absolute_error
35 |
36 | import os
37 | os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2" # TF 정보성 로그 숨김
38 | import tensorflow as tf
39 | from tensorflow import keras
40 | from tensorflow.keras import layers
41 |
42 | RNG_SEED = 42
43 | np.random.seed(RNG_SEED)
44 | tf.random.set_seed(RNG_SEED)
45 | BASE = pathlib.Path(__file__).resolve().parent.parent
46 | FIG_DIR, RES_DIR = str(BASE / "figures"), str(BASE / "results")
47 |
48 | # -----
49 | # 1. 데이터 로드 및 탐색
50 | # -----
51 | df = data("InstEval")
52 | print("데이터 크기:", df.shape) # (73421, 7)
53 | print(df.head())
54 | print("평점 분포:", df["y"].value_counts().sort_index().to_dict())
55 | # 1점 10,186 / 2점 12,951 / 3점 17,609 / 4점 16,921 / 5점 15,754 -> 최빈 클래스 비율 약 24%
56 |
57 | n_student = df["s"].nunique() # 2,972
58 | n_prof = df["d"].nunique() # 1,128
59 | print(f"학생 수 {n_student}, 교수 수 {n_prof}")
60 |
61 | # 평점 분포 시각화
62 | plt.figure(figsize=(6, 4))
63 | cnt = df["y"].value_counts().sort_index()
64 | plt.bar(cnt.index, cnt.values, color="steelblue")
65 |
66 | plt.xlabel("rating (1-5)"); plt.ylabel("count")
67 | plt.title(f"InstEval rating distribution (N={len(df):,})")
68 | plt.tight_layout()
69 | plt.savefig(f"{FIG_DIR}/p3_rating_dist.png", dpi=140)
```

```

69 | plt.close()
70 |
71 | # -----
72 | # 2. 전처리
73 | # -----
74 | # 레이블: 1~5점 -> 0~4 (sparse CE는 0부터 시작하는 정수 레이블을 받음)
75 | y = df["y"].to_numpy() - 1
76 |
77 | # 학생/교수 ID는 값에 빈 번호가 있으므로 0..(고유수-1)의 연속 정수로 다시 매김
78 | s_codes = df["s"].astype("category").cat.codes.to_numpy()
79 | d_codes = df["d"].astype("category").cat.codes.to_numpy()
80 |
81 | # 나머지 범주형 4개(학생 연차, 수강 시점, 필수 여부, 학과)는 카디널리티가 작아 원-핫 인코딩
82 | from sklearn.preprocessing import OneHotEncoder
83 | other = df[["studage", "lectage", "service", "dept"]].to_numpy()
84 | oh = OneHotEncoder(sparse_output=False).fit_transform(other) # (73421, 26)
85 | print("원-핫 특성 차원:", oh.shape[1])
86 |
87 | # 학습 64% / 검증 16% / 테스트 20% 분리 (클래스 비율 유지)
88 | idx = np.arange(len(y))
89 | idx_tmp, idx_te = train_test_split(idx, test_size=0.2, random_state=RNG_SEED, stratify=y)
90 | idx_tr, idx_va = train_test_split(idx_tmp, test_size=0.2, random_state=RNG_SEED, stratify=y[idx_tmp])
91 | print(f"train {len(idx_tr):,} / val {len(idx_va):,} / test {len(idx_te):,}")
92 |
93 | def take(ids):
94 |     """인덱스 배열로 (학생ID, 교수ID, 원-핫특성, 레이블) 묶음을 꺼내는 도우미"""
95 |     return [s_codes[ids], d_codes[ids], oh[ids], y[ids]]
96 |
97 | X_tr, y_tr = take(idx_tr)
98 | X_va, y_va = take(idx_va)
99 | X_te, y_te = take(idx_te)
100 |
101 | EPOCHS, BATCH = 12, 512
102 | EMB_DIM = 16 # 임베딩 차원: ID 하나를 16개의 실수로 표현
103 |
104 | # -----
105 | # 3. 모델 정의
106 | # -----
107 | def build_inputs():
108 |     """공통 입력층: 학생 ID(정수), 교수 ID(정수), 원-핫 특성(26차원)"""
109 |     in_s = layers.Input(shape=(), dtype="int32", name="student")
110 |     in_d = layers.Input(shape=(), dtype="int32", name="prof")
111 |     in_o = layers.Input(shape=(oh.shape[1],), name="others")
112 |     return in_s, in_d, in_o
113 |
114 | def build_baseline():
115 |     """[기준 모델] ID를 아예 쓰지 않고 원-핫 특성 26개만으로 소프트맥스 회귀.
116 |     '누가/누구를'이라는 정보가 없으면 어디까지 가능한지 하한선을 켜다."""
117 |     in_o = layers.Input(shape=(oh.shape[1],), name="others")
118 |     out = layers.Dense(5, activation="softmax")(in_o)
119 |     return keras.Model(in_o, out)
120 |
121 | def build_embedding_mlp(deep=False, act="relu"):
122 |     """[주 모델] 학생/교수 ID를 임베딩으로 변환해 MLP에 연결.
123 |     임베딩: 원-핫 벡터(2,972차원)에 가중치 행렬을 곱하면 결국 '행 하나를 꺼내는 것'과
124 |     같으므로, 거대한 원-핫 입력 대신 행렬에서 해당 행을 직접 꺼내 쓰는 중이다.
125 |     deep=True면 은닉층 6개를 쌓아 활성화 함수(act)에 따른 기울기 소실을 실험한다."""
126 |     in_s, in_d, in_o = build_inputs()
127 |     e_s = layers.Flatten()(layers.Embedding(n_student, EMB_DIM)(in_s)) # (16,)
128 |     e_d = layers.Flatten()(layers.Embedding(n_prof, EMB_DIM)(in_d)) # (16,)
129 |
130 |     z = layers.Concatenate()([e_s, e_d, in_o]) # 16+16+26 = 58차원
131 |     if deep:
132 |         for _ in range(6):
133 |             z = layers.Dense(64, activation=act)(z)
134 |     else:
135 |         z = layers.Dense(64, activation=act)(z)
136 |         z = layers.Dropout(0.3)(z) # 임베딩+MLP의 과적합 완화
137 |     out = layers.Dense(5, activation="softmax")(z)
138 |     return keras.Model([in_s, in_d, in_o], out)
139 |
140 | def fit_model(model, name, epochs=EPOCHS, features_only=False):
141 |     """공통 학습 함수: Adam + 크로스 엔트로피, 검증 곡선 기록.
142 |     features_only=True면 ID 입력 없이 원-핫 특성만 넣는다(기준 모델용)."""

```

```

142 |     xtr, xva, xte = (X_tr[2], X_va[2], X_te[2]) if features_only else (X_tr, X_va, X_te)
143 |     model.compile(optimizer="adam",
144 |                   loss="sparse_categorical_crossentropy",
145 |                   metrics=["accuracy"])
146 |     hist = model.fit(xtr, y_tr, validation_data=(xva, y_va),
147 |                     epochs=epochs, batch_size=BATCH, verbose=0)
148 |     _, te_acc = model.evaluate(xte, y_te, verbose=0)
149 |     print(f"{name}: test acc = {te_acc:.4f}")
150 |     return hist.history, float(te_acc), model
151 |
152 | # -----
153 | # 4. 학습 및 실험
154 | # -----
155 | # 다수결 기준선: 무조건 최빈 클래스(3점)로 찍었을 때
156 | maj = np.bincount(y_tr).argmax()
157 | acc_majority = float(np.mean(y_te == maj))
158 | print(f"다수결 기준선: {acc_majority:.4f}")
159 |
160 | hist_a, acc_a, _ = fit_model(build_baseline(), "A. 특성만(소프트맥스)", features_only=True)
161 | hist_b, acc_b, _ = fit_model(build_embedding_mlp(), "B. 임베딩+MLP (고정 12ep)")
162 |
163 | # B'. 같은 구조 + 조기 종료(early stopping):
164 | # B의 학습 곡선을 보면 검증 손실이 2~3 epoch에서 최저점을 찍고 다시 나빠진다(과적합).
165 | # 검증 손실이 3 epoch 연속 개선되지 않으면 멈추고, 가장 좋았던 시점의 가중치로 되돌린다.
166 | es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)
167 | best = build_embedding_mlp()
168 | best.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
169 | hist_b2_obj = best.fit(X_tr, y_tr, validation_data=(X_va, y_va),
170 |                       epochs=30, batch_size=BATCH, verbose=0, callbacks=[es])
171 | hist_b2 = hist_b2_obj.history
172 | _, acc_b2 = best.evaluate(X_te, y_te, verbose=0)
173 | acc_b2 = float(acc_b2)
174 | print(f"B'. 임베딩+MLP+조기종료: test acc = {acc_b2:.4f} (멈춘 epoch: {len(hist_b2['loss'])})")
175 |
176 | hist_c, acc_c, _ = fit_model(build_embedding_mlp(deep=True, act="relu"), "C. 깊은 망(ReLU)", epochs
177 |                               =10)
178 | hist_d, acc_d, _ = fit_model(build_embedding_mlp(deep=True, act="sigmoid"), "D. 깊은 망(sigmoid)", epo
179 |                               chs=10)
180 |
181 | # -----
182 | # 5. 평가: 정확도 + 순서형 지표 (MAE, 인접 허용 정확도)
183 | # 평점은 순서가 있는 레이블이므로 '몇 점 차이로 틀렸나'도 함께 본다
184 | # -----
185 | proba = best.predict(X_te, verbose=0)
186 | pred = proba.argmax(axis=1)
187 | mae = float(mean_absolute_error(y_te, pred)) # 평균 몇 점 차이인가
188 | acc_within1 = float(np.mean(np.abs(y_te - pred) <= 1)) # ±1점까지 맞다고 보면
189 | exp_pred = (proba * np.arange(5)).sum(axis=1) # 확률 가장 기대 평점 (연속값)
190 | mae_exp = float(mean_absolute_error(y_te, exp_pred))
191 | print(f"MAE {mae:.3f}, 기대평점 MAE {mae_exp:.3f}, ±1 정확도 {acc_within1:.4f}")
192 |
193 | cm = confusion_matrix(y_te, pred)
194 |
195 | # -----
196 | # 6. 시각화
197 | # -----
198 | # (1) 모델별 검증 정확도 곡선
199 | plt.figure(figsize=(7.5, 4.5))
200 | plt.axhline(acc_majority, color="gray", ls=":", label=f"majority ({acc_majority:.3f})")
201 | plt.plot(hist_a["val_accuracy"], label="A. features-only softmax")
202 | plt.plot(hist_b["val_accuracy"], label="B. embedding + MLP")
203 | plt.plot(hist_c["val_accuracy"], label="C. deep ReLU")
204 | plt.plot(hist_d["val_accuracy"], label="D. deep sigmoid")
205 | plt.xlabel("epoch"); plt.ylabel("validation accuracy")
206 | plt.title("Validation accuracy by model")
207 | plt.legend(fontsize=8); plt.tight_layout()
208 | plt.savefig(f"{FIG_DIR}/p3_model_compare.png", dpi=140)
209 | plt.close()
210 |
211 | # (2) 주 모델의 학습/검증 곡선 (과적합 관찰)
212 | fig, axes = plt.subplots(1, 2, figsize=(11, 4))
213 | axes[0].plot(hist_b["loss"], label="train loss")
214 | axes[0].plot(hist_b["val_loss"], label="val loss")

```

```

213 | axes[0].set_xlabel("epoch"); axes[0].set_ylabel("cross-entropy")
214 | axes[0].set_title("B. embedding+MLP loss"); axes[0].legend()
215 | axes[1].plot(hist_b["accuracy"], label="train acc")
216 | axes[1].plot(hist_b["val_accuracy"], label="val acc")
217 | axes[1].set_xlabel("epoch"); axes[1].set_ylabel("accuracy")
218 | axes[1].set_title("B. embedding+MLP accuracy"); axes[1].legend()
219 | plt.tight_layout()
220 | plt.savefig(f"{FIG_DIR}/p3_main_curve.png", dpi=140)
221 | plt.close()
222 |
223 | # (3) 혼동행렬: 틀려도 이웃 점수로 틀리는지 확인
224 | plt.figure(figsize=(5.5, 5))
225 | plt.imshow(cm, cmap="Blues")
226 | plt.colorbar()
227 | for i in range(5):
228 |     for j in range(5):
229 |         plt.text(j, i, cm[i, j], ha="center", va="center", fontsize=8,
230 |                 color="white" if cm[i, j] > cm.max()/2 else "black")
231 | plt.xticks(range(5), [1,2,3,4,5]); plt.yticks(range(5), [1,2,3,4,5])
232 | plt.xlabel("predicted rating"); plt.ylabel("actual rating")
233 | plt.title("Confusion matrix (B' embedding+MLP, early stopping)")
234 | plt.tight_layout()
235 | plt.savefig(f"{FIG_DIR}/p3_confusion.png", dpi=140)
236 | plt.close()
237 |
238 | # -----
239 | # 7. 결과 저장
240 | # -----
241 | results = {
242 |     "n_samples": int(len(y)),
243 |     "n_student": int(n_student), "n_prof": int(n_prof),
244 |     "split": {"train": len(idx_tr), "val": len(idx_va), "test": len(idx_te)},
245 |     "rating_counts": df["y"].value_counts().sort_index().tolist(),
246 |     "test_acc": {
247 |         "majority": acc_majority,
248 |         "A_features_only": acc_a,
249 |         "B_embedding_mlp_12ep": acc_b,
250 |         "B2_early_stopping": acc_b2,
251 |         "C_deep_relu": acc_c,
252 |         "D_deep_sigmoid": acc_d,
253 |     },
254 |     "B2_stopped_epoch": len(hist_b2["loss"]),
255 |     "B_final": {"train_acc": hist_b["accuracy"][-1], "val_acc": hist_b["val_accuracy"][-1],
256 |                 "train_loss": hist_b["loss"][-1], "val_loss": hist_b["val_loss"][-1]},
257 |     "mae": mae, "mae_expected": mae_exp, "acc_within1": acc_within1,
258 |     "confusion_matrix": cm.tolist(),
259 | }
260 | print(json.dumps(results, indent=2))
261 | with open(f"{RES_DIR}/p3_results.json", "w") as f:
262 |     json.dump(results, f, indent=2)
263 |
264 | print("프로젝트 3 완료")
265 |

```